

Síntesis, traducción y verificación de sistemas de ecuaciones utilizados en Protocolos de Conocimiento Cero

Lucía Martínez Martínez

Grado en Ingeniería Informática
Universidad Complutense de Madrid



Trabajo de Fin de Grado
Curso 2025 - 2026

Directores:
Clara Rodríguez Núñez
Alejandro Hernández Cerezo

Índice general

1. Introducción a Circom	3
1.1. Zero Knowledge Proof	3
1.2. CIRCOM	4
1.3. R1CS	7
1.4. Snarkjs	7
1.5. Preámbulo del problema	8
2. Desarrollo del problema	9
2.1. Definición del problema	9
2.2. Ejemplo del problema	12
2.3. Notación	15
2.4. Reglas de transformación	16
2.4.1. Operadores Lógicos	17
2.4.2. Operadores de Comparación	17
2.4.2.1. Tratamiento de número negativos	17
2.4.2.2. Tamaño de los números comparados	19
2.4.2.3. Operadores $<, \leq, >$ y $\geq$	20
2.4.2.4. Operadores $=$ y $\neq$	22
2.4.2.5. Traducción de operadores	22
2.4.3. Operadores Aritméticos	24
2.4.4. Operadores de desplazamiento	24
2.4.5. Acceso a Arrays	24
2.4.6. Expresiones condicionales	24
2.4.7. Expresiones condicionales	24
2.5. Ejemplo con constraints generadas automáticamente	24
3. Implementación	25
3.1. Explicación y ejemplo de la construcción de las reglas	25
4. Experimentos	26
4.1. Aplicación de la generación automática sobre juegos simples	26
4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints	26
4.3. Caso de uso, verificación y búsqueda de bugs	26

Introducción a Circom

1.1. Zero Knowledge Proof

En términos criptográficos, una prueba de conocimiento cero, conocida como Zero Knowledge Proof (o sus siglas inglesas ZKP), es una **familia de protocolos** que permite el establecimiento de métodos cuyo objetivo es permitir que una parte (**prover**) demuestre a otra (**verifier**) la validez de una afirmación **sin revelar información sobre la misma**, más allá de su validez.

Alejandro (prover) tiene el décimo de la lotería de Navidad que ha sido premiado y quiere demostrarle a **Clara (verifier)** que posee el número ganador, pero sin que ésta sea conocedora del mismo (y así evitar que pueda cobrar el premio o se lo robe). Para ello, utilizan una caja fuerte que solo se abre introduciendo la combinación del decimo premiado.

Clara escribe en un papel un dígito aleatorio, lo introduce en la caja y la cierra. Posteriormente, Clara se gira, de forma que no ve nada más de la caja.

Alejandro, que conoce el número del décimo, introduce la combinación, abriendo así la caja y leyendo el número escrito por Clara en el papel en voz alta. En ese momento, Clara sabe que ha podido abrir la caja, y por tanto, obtiene una prueba de que Alejandro es conocedor de esa información.

Sin embargo, Clara podría sospechar que Alejandro ha adivinado el dígito al azar, ya que la probabilidad es del 10 %, por lo que Clara decide repetir la prueba varias veces.

No obstante, si observamos $0,1 * 0,1 = 0,01$, se reduce la probabilidad inicial de 0,1 a 0,01. Es decir, a medida que las repeticiones aumentan, menor es la probabilidad de que se acierte al azar, ya que ésta converge a 0, alcanzando así una certeza práctica sin haber revelado ni un solo dígito del décimo.

Ejemplo 1.1: Zero Knowledge Proof

Como hemos visto en el *Ejemplo 1.1*, el concepto de **ZKP** queda ilustrado de una forma muy intuitiva. Sin embargo, para trasladar esta lógica al contexto criptográfico, es necesario formalizar estas interacciones mediante protocolos específicos.

Históricamente, el término de ZKP apareció en los años 80 como un concepto puramente teórico, hasta su desarrollo en la práctica. Dicha noción surgió con la intención de reforzar tanto la seguridad como la privacidad en el ámbito criptográfico.

La **criptografía** constituye el pilar fundamental de la seguridad en el mundo digital, permitiendo proteger información sensible frente a amenazas. Las técnicas criptográficas nos ofrecen la capacidad de garantizar confidencialidad así como un equilibrio entre transparencia y privacidad.

Gracias a este equilibrio, las técnicas criptográficas pudieron llevar a la práctica el concepto teórico ZKP en protocolos prácticos. Actualmente, esto ha llevado a la creación de grandes familias de protocolos, destacando especialmente **zk-SNARKs**.

Succinct Non-Interactive Argument of Knowledge, mayormente conocido por sus siglas **zk-SNARKs**, destacan por su reducido tamaño de prueba y corto tiempo de verificación. Sin embargo, presentan la desventaja de requerir una fase inicial conocida como **configuración de confianza**.

Definición 1.1: zk-SNARKS

Una configuración de confianza es una fase en la que se produce la generación de valores aleatorios que deben destruirse de forma inmediata. Si estos valores aleatorios son expuestos, se compromete la seguridad del sistema.

Definición 1.2: Configuración de confianza

El motivo de que estos números aleatorios deban borrarse inmediatamente es debido a que cualquier entidad conocedora de esos valores podría almacenarlos y así generar **pruebas falsas**, de forma que se podría vulnerar la seguridad del sistema.

Dada la complejidad que implica implementar estos protocolos desde cero, se han desarrollado lenguajes de programación específicos denominados **Domain Specific Languages**, conocidos como DSL.

El objetivo es que los desarrolladores que no son expertos en criptografía puedan programar las declaraciones que desean demostrar. Es en este contexto donde cobra importancia **Circom**, una herramienta creada con el fin de facilitar la creación de circuitos y su posterior transformación en sistemas de restricciones.

1.2. CIRCOM

Circom es un lenguaje de programación basado en la descripción de circuitos (DSL), cuyo objetivo principal no es el cálculo, sino definir un sistema en el que las **señales** se relacionan entre sí.

Una señal es un componente que actúa como cable dentro del circuito, el cual transporta valores a través de las puertas lógicas del mismo y que forma parte de las

ecuaciones que el programador desea verificar. Estas señales constituyen las restricciones que formarán parte del archivo **R1CS**.

Esta visión es fundamental, ya que las ZKP no pueden interpretar código directamente; requieren que la computación se descomponga en una estructura de restricciones algebraicas denominada R1CS, que detallaremos en la sección **1.3. R1CS**.

Además de su finalidad criptográfica, es importante entender la diferencia entre un lenguaje basado en circuitos a uno de propósito general, siendo Circom basado en circuitos donde la estructura del mismo debe ser estática; esto implica que cualquier valor determinante en la estructura del circuito (como tamaños de arrays, número de iteraciones de un bucle, etc..) **debe conocerse en tiempo de compilación**. Esto se debe a que así aseguramos que el sistema de restricciones R1CS que se genera en base a este circuito, sea fijo.

El proceso de compilación de Circom refleja un paradigma híbrido entre lo **declarativo e imperativo**. De esta forma, el compilador genera dos archivos esenciales para generar la prueba:

- **Archivo R1CS:** Contiene el conjunto de todas las restricciones algebraicas que definen la lógica del circuito. Estas restricciones son añadidas desde un programa Circom por medio de los símbolos `<==` y `===`.
- **Generador de Testigo (Witness):** Un ejecutable (en formato C++ o WASM) encargado de calcular los valores de todas las señales del circuito, incluyendo los valores intermedios, a partir de las entradas proporcionadas. Estas asignaciones son añadidas desde un programa Circom por medio de los símbolos `<==` y `<--`

Partiendo de estos dos archivos, podemos generar la prueba por medio de **Snarkjs**. Esta herramienta toma el archivo R1CS y el Witness obtenidos tras la compilación de un programa Circom para generar la prueba asociada. Analizaremos en detalle su funcionamiento y comandos específicos en la sección **1.4. Snarkjs**.

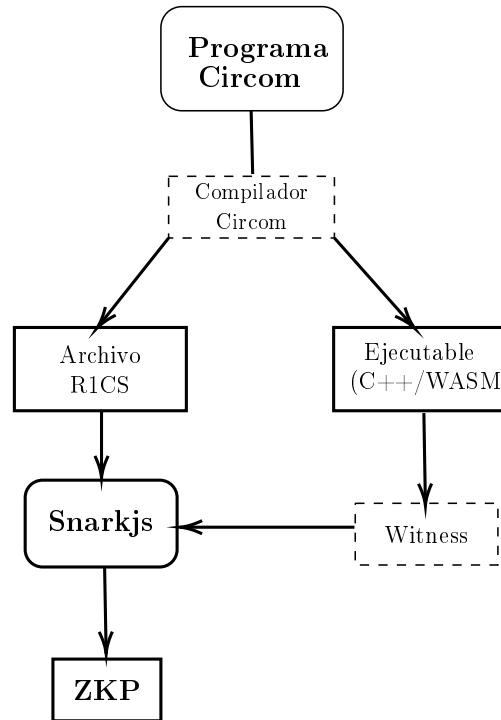
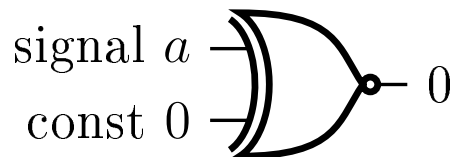


Figura 1.1: Proceso de transformación a ZKP desde Circom

Como hemos mencionado anteriormente, las señales son las restricciones establecidas por el programador, las cuáles quiere verificar. A continuación se muestra un ejemplo de un programa Circom y su representación en un circuito lógico, en el que se añade la restricción $a = 0$.

```

1      template var_zero(){
2          signal input a;
3          a === 0;
4      }
5
6      main component = var_zero();
  
```



Ejemplo 1.2: Archivo var_zero.circom

El ejemplo anterior refleja como Circom transforma la ecuación $a === 0$ en un circuito lógico equivalente. De esta forma podemos ver que las entradas del componente lógico XOR son a y 0 , forzando a que la salida sea 0 .

El funcionamiento de una puerta lógica XOR se resume en que la salida es 0 si ambas entradas son iguales. Por tanto, al forzar que la salida sea 0 , obligamos de esta forma a que $a = 0$ se cumpla.

1.3. R1CS

Rank-1 Constraint System (conocido como **R1CS**), es un sistema de representación matemático diseñado para transformar tareas computacionales complejas en un conjunto de restricciones algebraicas. Su relevancia incide en la propiedad de que generar la solución a un problema puede ser caro, mientras que su **verificación es muy eficiente y rápida**, clave en las pruebas de conocimiento cero.

R1CS impone restricciones matemáticas complejas. Las ecuaciones, aparte de no poder exceder el segundo grado (*condición necesaria, pero no suficiente*), cada restricción debe poder expresarse como el producto de dos combinaciones lineales de variables, teniendo como resultado una tercera combinación lineal:

$$A \cdot B - C = 0$$

Donde A , B y C son combinaciones lineales de las señales del circuito. En términos prácticos, esto implica que una sola restricción solo puede contener una multiplicación entre variables. Operaciones que incluyan múltiples productos independientes, aunque sean de segundo grado, deben ser descompuestas en varias restricciones consecutivas por medio de variables auxiliares con el fin de cumplir con este estándar.

Siguiendo el *Ejemplo 1.2*, la restricción $a == 0$ se traduce al formato R1CS como el producto de dos combinaciones lineales A y B , forzando así que el valor de la señal a sea cero.

Dada la declaración:

$$(a) \times (1) = (0)$$

Donde la estructura $A \cdot B = C$ se satisface asignando:

- $A = a$ (Combinación lineal 1)
- $B = 1$ (Combinación lineal 2)
- $C = 0$ (Resultado de la restricción)

Ejemplo 1.3: Transformación de `var_zero.circom` a formato R1CS

Gracias a R1CS, es posible que el código **escrito en Circom pueda transformarse en un sistema de restricciones**, consiguiendo así que la verificación sea sumamente eficiente.

1.4. Snarkjs

Snarkjs es una librería de JavaScript creada para la generación y verificación de pruebas de conocimiento cero partiendo de las restricciones en formato R1CS correspondientes, que puede ser ejecutado tanto en dispositivos como en servidores con *Node.js*, materializando el concepto de *1.1 zk-SNARKS* nombrado anteriormente.

Como se mostró previamente en la *1.1 Figura*, Snarkjs requiere de dos archivos para generar la prueba:

- *Witness*, generado por el ejecutable C++/WASM.
- *Archivo R1CS*, restricciones de la lógica del circuito.

Una vez disponibles estos dos archivos y la *1.2 Configuración de confianza* finalizada, Snarkjs genera la prueba de conocimiento cero que permite que el Prover pueda demostrarle al Verifier la validez de su declaración.

1.5. Preámbulo del problema

Circom es un lenguaje de programación cuyo aprendizaje y aplicación puede ser tedioso en sus inicios, sobre todo si no se tiene habilidad tratando con lenguajes de bajo nivel.

Recapitulando el comportamiento *imperativo-declarativo* de Circom, este generaba dos archivos: R1CS para las restricciones y Witness para las asignaciones de señales.

El símbolo `<--` permite añadir una asignación al archivo Witness, pero no al R1CS, ya que este es exclusivo para las restricciones del programa (añadidas con los operadores `<==` y `===`).

Aprovechando la flexibilidad de las asignaciones y la sintaxis ofrecida por la naturaleza *declarativa-imperativa* de Circom, podemos implementar una nueva funcionalidad realizando modificaciones en su compilador, las cuales permiten traducir la lógica programada utilizando las señales añadidas al Witness (por medio de `<--`) directamente en restricciones de Circom.

De este modo, **el compilador actuaría como un puente**, transformando automáticamente el flujo imperativo en un sistema de restricciones.

Desarrollo del problema

2.1. Definición del problema

La problemática principal que surge durante el proceso de aprendizaje de Circom reside sustancialmente en su diferencia respecto a lenguajes de alto nivel, tanto en objetivo como restricciones de uso.

Circom, como se ha mencionado previamente, es un lenguaje de bajo nivel, presentando dificultades en su aprendizaje debidas al gran cambio de paradigma respecto a otros lenguajes de programación, más comunes entre los usuarios.

El cambio a un lenguaje basado en circuitos requiere una conversión hacia otra mentalidad, tanto para comprender cuál es el objetivo real de su naturaleza basada en circuitos, así como adaptarse a los recursos ofrecidos por el lenguaje.

Por tanto, se presenta un doble reto:

1. **Asimilar el paradigma de verificación frente al de ejecución**, comprender que el objetivo principal no es realizar un cálculo o cómputo, sino generar un sistema de restricciones con el objetivo de verificar la validez de una demostración.
2. **La restricción de recursos y diseño algebraico**, implicando diseñar circuitos donde cada operación debe ser expresada mediante restricciones algebraicas, garantizando que cualquier solución sea verificable así como asumir la naturaleza estática del circuito.

De esta manera, puede observarse que Circom es un lenguaje que no presenta una curva de aprendizaje elevada, sino que requiere de tiempo y dedicación para desarrollar habilidad y destreza.

Por tanto, para aquellas personas que requieran el uso de este lenguaje, pero se enfrenten a la dificultad del cambio de paradigma, aún teniendo experiencia en lenguajes imperativos, se expone una solución cuyo objetivo es facilitar el uso de Circom, explotando su cualidad imperativa.

La propuesta radica en la modificación del compilador de Circom para incorporar una nueva funcionalidad que habilita la traducción del cuerpo imperativo desarrollado en un programa Circom a ecuaciones algebraicas en formato R1CS equivalentes.

El cuerpo imperativo tras la modificación del compilador no sería únicamente funcional para la generación del testigo, sino que con dicha propuesta se permitiría la conversión del código correspondiente tal y como se haría si dicho cuerpo fuera el equivalente al generador de las restricciones R1CS.

Compilador sin modificación

```
template funcion() {  
    signal input a;  
    signal output b;
```

```
    if(a == 0){  
        b <-- 0;  
    }  
    else{  
        b <-- 1;  
    }  
}
```

Código parte del generador del testigo, no genera restricciones y por tanto, no forma parte del archivo R1CS

```
    component iz = IsZero();  
    iz.in <== a;  
    b <== (1 - iz.out);  
}
```

Código generador del archivo R1CS con las restricciones del circuito

}

Compilador con modificación

```
template funcion() autocomplete{  
    signal input a;  
    signal output b;
```

Cabecera que indica al compilador que se quiere usar la nueva funcionalidad

```
    if(a == 0){  
        b <-- 0;  
    }  
    else{  
        b <-- 1;  
    }  
}
```

Tras la modificación, su único fin no será formar parte del generador de testigo. Ahora, el compilador traducirá este código a restricciones. Estas restricciones serán equivalentes a las del bloque de código inferior, ya que son fragmentos de código equivalentes.

```
    component iz = IsZero();  
    iz.in <== a;  
    b <== (1 - iz.out);  
}
```

Restricciones algebraicas del cuerpo imperativo

|||

Restricciones algebraicas del cuerpo declarativo

Ejemplo 2.1: Implicaciones de la nueva funcionalidad

De esta forma, Circom sería un lenguaje de programación mucho mas accesible para los usuarios al facilitar su uso, gracias a la incorporación esta nueva funcionalidad.

Para ilustrar de forma práctica las limitaciones mencionadas y la dificultad que implica la definición manual de restricciones, se presenta a continuación un ejemplo que contrasta el flujo actual de Circom frente a la solución propuesta.

2.2. Ejemplo del problema

Una manera más precisa de materializar esta propuesta es por medio de un ejemplo, utilizando para ello un programa real en Circom.

El programa que utilizaremos como ejemplo es el clásico juego de *piedra, papel y tijera*. Éste consiste en 3 reglas principales.

1. Piedra gana a Tijera, pero pierde contra Papel
2. Papel gana a Piedra, pero pierde contra Tijera.
3. Tijera gana contra Papel, pero pierde contra Piedra.

Este juego tan característico y conocido se debe en gran parte a su simplicidad. Sin embargo, transformar su lógica en un programa Circom que lo materialice en un circuito es una tarea más tediosa y compleja.

A continuación, se muestra un programa Circom equivalente a la lógica de este juego.

```
1     template piedra_papel_tijera () {
2
3         // Piedra -> 0, Papel -> 1, Tijera -> 2
4         signal input jugador1; // Representa el estado del
           jugador 1
5         signal input jugador2; // Representa el estado del
           jugador 2
6         signal output ganador; // 0 -> jugador1; 1 -> jugador2,
           2 -> empate
7
8         signal eq;
9         signal inv;
10        signal out;
11        signal j1;
12        signal j2;
13
14        // --- Comparaciones de cada jugador y estado ---
15
16        // Jugador 1
17        component J1_0 = igualdadRepetida(0);
18        J1_0.in <== jugador1;
19
```

```

20     component J1_1 = igualdadRepetida(1);
21     J1_1.in <== jugador1;
22
23     component J1_2 = igualdadRepetida(2);
24     J1_2.in <== jugador1;
25
26     // Jugador 2
27     component J2_0 = igualdadRepetida(0);
28     J2_0.in <== jugador2;
29
30     component J2_1 = igualdadRepetida(1);
31     J2_1.in <== jugador2;
32
33     component J2_2 = igualdadRepetida(2);
34     J2_2.in <== jugador2;
35
36     // Igualdad entre jugadores (empate)
37     signal eq0;
38     signal eq1;
39     signal eq2;
40
41     eq0 <== (J1_0.out) * (J2_0.out);
42     eq1 <== (J1_1.out) * (J2_1.out);
43     eq2 <== (J1_2.out) * (J2_2.out);
44     eq <== eq0 + eq1 + eq2;
45
46     // Inversion de empate
47     inv <== (1 - eq);
48
49     // Combinaciones ganadoras Jugador 1
50     signal j1_0;
51     signal j1_1;
52     signal j1_2;
53
54     j1_0 <== (J1_0.out) * (J2_2.out);
55     j1_1 <== (J1_1.out) * (J2_0.out);
56     j1_2 <== (J1_2.out) * (J2_1.out);
57
58     j1 <== j1_0 + j1_1 + j1_2;
59
60     // Combinaciones ganadoras Jugador 2
61     signal j2_0;
62     signal j2_1;
63     signal j2_2;
64
65     j2_0 <== (J1_0.out) * (J2_1.out);
66     j2_1 <== (J1_1.out) * (J2_2.out);
67     j2_2 <== (J1_2.out) * (J2_0.out);
68

```

```

69     j2 <== j2_0 + j2_1 + j2_2;
70
71     // Calculo final del ganador
72     out <== j2; // Simplificado: j2*1 + j1*0 es igual a j2
73
74     ganador <== ((1 - inv) * 2) + (out * inv);
75 }

```

Código 2.1: Implementación declarativa de Piedra, papel y tijera en Circom

Tras la introducción del código equivalente a la lógica del juego, puede apreciarse a simple vista la complejidad de describir flujos lógicos usando Circom, requiriendo tanto de maestría en su sintaxis y recursos, como la destreza para reflejar fielmente la lógica del juego.

Sin embargo, la incorporación de nuestra modificación sobre el compilador simplificará este proceso, permitiendo una programación más intuitiva sin privar al lenguaje de las restricciones algebraicas necesarias para generar la prueba de conocimiento cero.

Dicha incorporación simplifica considerablemente el flujo lógico a describir, ya que se permite más libertad en cuanto al uso de recursos, habilitando al usuario a utilizar componentes como los condicionales, que precisamente para la descripción de la lógica de este juego reduce la complejidad.

Esto evita que el programador tenga que transformar manualmente cada bifurcación del algoritmo en expresiones algebraicas, delegando ese trabajo al compilador modificado.

```

1  template piedra_papel_tijera () autocomplete{
2
3      // Piedra -> 0, Papel -> 1, Tijera -> 2
4      signal input jugador1; // Representa el estado del jugador
5      signal input jugador2; // Representa el estado del jugador
6      signal output ganador; // 0 -> jugador1; 1 -> jugador2, 2
7      -> empate
8
9      if(jugador1 == jugador2){ //Empate
10         ganador <-- 2;
11     }
12     else if(jugador2 == 0 && jugador1 == 2){ //Gana jugador 2
13         con piedra
14         ganador <--1;
15     }
16     else if(jugador1 == 0 && jugador2 == 2){ // Gana jugador 1
17         con piedra
18         ganador <--0;
19     }
20 }

```

```

17     else if(jugador2 == 2 && jugador1 == 1){ //Gana jugadpr 2
18         con tijera
19         ganador <-- 1;
20     }
21     else if(jugador1 == 2 && jugador2 == 1){ //Gana jugador 1
22         con tijera
23         ganador <-- 0;
24     }
25     else if(jugador2 == 1 && jugador1 == 0){ //Gana jugador 2
26         ocn papel
27         ganador <-- 1;
28     }
29     else if(jugador1 == 1 && jugador2 == 0){ // Gana jugador 1
30         con papel
31         ganador <-- 0;
32     }
33     else{
34         ganador <-- 3; //Valor no valido
35     }
36 }

```

Código 2.2: Implementación imperativa de Piedra, papel y tijera en Circom

Tras analizar ambos los bloques de código lógicamente equivalentes, es evidente la simplicidad, legibilidad e independencia que la nueva funcionalidad incorpora al lenguaje.

Si bien este beneficio es ya notable en un ejemplo sencillo como este, su verdadero potencial reside en la migración a problemas más complejos.

Al trasladar esta lógica a sistemas de mayor envergadura, los beneficios en términos de mantenibilidad y reducción de errores en el diseño de circuitos como una mayor accesibilidad pueden ser exponenciales.

2.3. Notación

A lo largo del documento estudiaremos cómo traducir instrucciones y expresiones generadas por un lenguaje imperativo simple en un conjunto de ecuaciones en formato R1CS equivalente definiremos las siguientes funciones:

Definimos la función de traducción de expresiones $TC : Expr \rightarrow C \times Expr$ siendo:

- C un sistema de ecuación en formato R1CS
- C es una expresión lineal

Definición 2.1: Notación

La intuición es que en caso de que las constraints C se verifiquen, entonces e y e' siempre tienen el mismo valor.

Vamos a definir la función TC de forma inductiva, comenzando por los casos base (números y variables), para luego pasar a definir los casos compuestos.

Casos base

- Si la expresión es un número n :

$$TC(n) = TC(\emptyset, n)$$

- Si la expresión es una variable v :

$$TC(v) = TC(\emptyset, v)$$

2.4. Reglas de transformación

En esta sección se encuentra la documentación relativa a la implementación de las reglas de transformación pertinentes para hacer posible la migración del problema descrito a la práctica.

2.4.1. Operadores Lógicos

Operación	Deducción
$e = \text{not } b$	$\frac{TC(e_1) = (C, e'_1)}{TC(not\ e_1) = ((e'_1) * (e'_1 - 1) == 0) \wedge (C, 1 - e'_1)}$
$e = \text{and } b$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ and } e_2) = \left(\begin{array}{c} C_1 \wedge C_2 \wedge \\ (e'_1 * (e'_1 - 1) = 0) \wedge (e'_2 * (e'_2 - 1) = 0) \\ \wedge \{v_{aux} = e'_1 * e'_2\}, v_{aux} \end{array} \right)}$
$e = \text{or } b$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ or } e_2) = \left(\begin{array}{c} C_1 \wedge C_2 \wedge \\ (e'_1 * (e'_1 - 1) = 0) \wedge (e'_2 * (e'_2 - 1) = 0) \\ \wedge \{v_{aux} = e'_1 * e'_2\}, e'_1 + e'_2 - v_{aux} \end{array} \right)}$
$e = e_1 \& e_2$	$\frac{TC(e_1) = (C_1, e_{1-\text{binario}}) \quad TC(e_2) = (C_2, e_{2-\text{binario}})}{TC(e_1 \& e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e_{1-\text{binario}}[i] * e_{2-\text{binario}}[i]\}, v_{aux})}$
$e = e_1 \mid e_2$	$\frac{TC(e_1) = (C_1, e_{1-\text{binario}}) \quad TC(e_2) = (C_2, e_{2-\text{binario}})}{TC(e_1 \mid e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e_{1-\text{binario}}[i] * e_{2-\text{binario}}[i]\}, e_{1-\text{binario}}[i] + e_{2-\text{binario}}[i] - v_{aux})}$

2.4.2. Operadores de Comparación

2.4.2.1. Tratamiento de número negativos

Un detalle destacable en la función *LessThanEq()* mencionada a continuación, es que para números negativos podría dificultar su uso.

El motivo reside en trabajamos sobre un cuerpo finito $\mathbb{F}_p$, de forma que los números negativos realmente no existen. En su lugar, se representan mediante el módulo del campo.

Por ejemplo, el valor -1 se interpreta como $p-1$, un número extremadamente grande que se aproxima al orden del primo del campo.

Para solventar este problema, recurrimos a las transformación de los números a comparar de sistema decimal a binario. Esta conversión es estrictamente necesaria

para asegurar que los números correspondientes no exceden el rango especificado.

La función $Num2Bits(n)$ consigue cumplir esta precondition ya que su funcionamiento radica en la transformación numérica de decimal a binario, reconstruyendo el número decimal entrante durante la misma.

De esta forma, al terminar la transformación y antes de salir de la función, se realiza una comprobación para verificar si el número decimal entrante y el número decimal reconstruido durante la transformación son iguales.

Debido al excesivo tamaño del número entrante, nunca se cumplirá la restricción de que ambos sean iguales debido a que el tamaño máximo fijado (N) es siempre menor que el tamaño del número negativo, desencadenando que la reconstrucción no realizará un bucle de tamaño mayor a al número N fijado.

Por tanto, no habrá un bucle que reconstruya un número decimal negativo ya que nunca se realizan las iteraciones necesarias, forzando de esta forma a que solo los números positivos cumplan dicha condición.

Intuición del funcionamiento de $Num2Bits(n)$ con un número negativo entrante:

1. El número negativo se interpreta como el módulo del campo.
2. Éste se interpretará como un número superior a $2^n - 1$, saliéndose fuera del rango $[0, 2^n - 1]$, por tanto la restricción $lc1 === in$ de la función nunca se cumplirá en este caso y la prueba no se generará.

En conclusión, $Num2Bits()$ garantiza el uso de número positivos dentro del rango mencionado de manera implícita.

Observación: Gracias a la constraint $lc1 === in$ (fuerza a que la entrada cumpla el rango $[0, 2^n - 1]$), podemos evitar el uso de número negativos, ya que estos no cumplirán tal restricción.

Num2Bits(n):

```
1      template Num2Bits(n){
2          signal input in;
3          signal output out[n];
4          var lc1=0; // Reconstruye el numero decimal a
                    // convertir para la constraint
5
6          var e2=1; // Evitar la potencia
7          for (var i = 0; i<n; i++) {
8              out[i] <-- (in >> i) & 1; // Contruimos el numero
                    // en binario
9              out[i] * (out[i] -1 ) == 0;
10             lc1 += out[i] * e2;
11             e2 = e2+e2;
12         }
13
14         lc1 == in; // Constraint mencionada anteriormente
15     }
```

Código 2.1: Función Num2Bits(n)

2.4.2.2. Tamaño de los números comparados

El tamaño de los números comparados es un factor importante, ya que dicho tamaño puede ser por un lado conocido e igual, mientras que por otro lado puede ser desconocido o de diferentes tamaños.

Para resolver este problema, vamos a definir un complemento opcional por cada operador de comparación existente.

Dicho complemento consiste en restringir el tamaño de los números comparados a un tamaño especificado por el usuario. Si el usuario no hace uso de este complemento opcional para la especificación del tamaño de los números a comparar, se utilizará el tamaño máximo por defecto, correspondiente a 253.

Operador $\leq$

- $\leq$: Comparaciones de cuyos números se conoce el tamaño y además tienen el mismo tamaño.
- La llamada a la función sería *lessThan*(253) para estos casos, ya que el tamaño máximo es 253 en el cuerpo finito.

Operador $\leq _ (n)$

- $\leq_n$: Comparaciones de cuyos números no se conoce el tamaño y/o no tiene el mismo tamaño.
- La llamada a la función sería *lessThan*(*n*) para estos casos.

Ejemplo 2.2: Complemento de tamaño para operador $\leq$

Este complemento aplica para todos los operadores de comparación, siguiendo la misma estructura que la del *Ejemplo 2.2* aplica sobre el operador $\leq$;

2.4.2.3. Operadores $<, \leq, >$ y $\geq$

Para la implementación de dichos operadores únicamente es necesaria la función *SecureLessThanEq*(*n*).

El motivo reside en que una comparación entre una variable *a* y *b*, con *SecureLessThanEq*(*n*) podemos conocer si $a \leq b$ o bien $a > b$ en caso contrario (*GreaterThan*()).

Siguiendo esta lógica, podemos usar la misma función invirtiendo el orden de los parámetros, consiguiendo de esta manera los operadores *GreatherThanEq*() y *LessThan*() si $b \leq a$ o $b > a$.

La implementación de *SecureLessThanEq*(*n*), aparte de utilizar la función *Num2Bits*() para cada par de variables a comparar, aplica Complemento a 2.

Es decir, dadas las variables *A* y *B*, previamente transformadas a bits, aplicamos $(\text{NOT } A) + B + 1$. La inversión de los bits de *A* y la suma tanto de *B* como de 1 es equivalente a la operación $B - A$.

Por tanto, cuando el minuendo es mayor o igual que el sustraendo, la suma de los bits junto con el complemento a dos produce un acarreo (overflow) en la posición *n* (el bit extra, bit más significativo). Por otro lado, si el minuendo es menor que el sustraendo, no se produce dicho overflow.

De esta forma sabemos que si el bit de acarreo es 1, podemos concluir que $B \geq A$,

ya que al hacer la suma con Complemento a 2, tenemos que:

$$\begin{aligned}
 & (\text{NOT } A) + B + 1 \\
 & [(2^n - 1) - A] + B + 1 \\
 & 2^n - 1 - A + B + 1 \\
 & 2^n - A + B \\
 & B + (2^n - A)
 \end{aligned} \tag{2.1}$$

Como podemos ver en las ecuaciones anteriores, debido al requerimiento de $2^n - 1$ por parte de NOT A, es vital el +1 de $(\text{NOT } A) + B + 1$. De lo contrario el resultado de la resta estaría desplazado por una unidad al no eliminar ese -1 de NOT A.

Partiendo de $2^n + B - A$, si B es mayor o igual que A, entonces $(B-A) \geq 0$. De este modo, tendríamos que $2^n + (\text{num.positivo}) \geq 2^n$, concluyendo que si $B \geq A$ obtendríamos siempre un resultado mayor o igual a 2^n , excediendo siempre los n bits y como consecuencia, produciendo un overflow.

Siguiendo la misma lógica, si el bit de acarreo es 0 significa que A es mayor que B ya que $B-A < 0$, y de esta forma obtendríamos $2^n + (\text{num.negativo}) < 2^n$, concluyendo así que no podría producirse un overflow.

```

template SecureLessThan(n){
    signal input in[2];
    signal output out;

    //-----

    component checkA = Num2Bits(n);
    checkA.in <== in[0];

    component checkB = Num2Bits(n);
    checkB.in <== in[1];

    adder = BinSum(n, 2);

    var i;
    for (i=0;i<n;i++) {
        checkA.out[i] ==> adder.in[0][i];
        checkB.out[i] ==> adder.in[1][i];
    }

    // Miramos el bit + significativo (
    // MSB), 1 si A < B, 0 de lo
    // contrario
    adder.out[n-1] ==> out;
}

```

Con esta pequeña modificación, al usar para ambos números a comparar la función *Num2Bits()* nos aseguramos de que se cumpla la restricción de ser números enteros positivos dentro del rango.

2.4.2.4. Operadores $=$ y $!=$

2.4.2.5. Traducción de operadores

A continuación, definiremos para cada operador su deducción.

Operación	Deducción
$e = e_1 = e_2$	$\frac{\text{signal diff} \leq e_1 - e_2 \quad TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 = e_2) = (C_1 \wedge C_2 \wedge \{IsZero().in = diff, IsZero().out = v_{aux}\}, v_{aux})}$
$e = e_1 \neq e_2$	$\frac{\text{signal diff} \leq e_1 - e_2 \quad TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \neq e_2) = (C_1 \wedge C_2 \wedge \{IsZero().in = diff, IsZero().out = v_{aux}\}, 1 - v_{aux})}$
$e = e_1 < e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 < e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThan(253).in1 = e'_1, \\ SecureLessThan(253).in2 = e'_2, \\ SecureLessThan(253).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 <_n e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 <_n e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThan(n).in1 = e'_1, \\ SecureLessThan(n).in2 = e'_2, \\ SecureLessThan(n).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 \leq e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThanEq(253).in1 = e'_1, \\ SecureLessThanEq(253).in2 = e'_2, \\ SecureLessThanEq(253).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 \leq_n e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq_n e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThanEq(n).in1 = e'_1, \\ SecureLessThanEq(n).in2 = e'_2, \\ SecureLessThanEq(n).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 > e_2$	

Nota: Las funciones `IsZero()` son circuitos que comparan si un valor es igual a 0.
Nota: Las funciones `IsEqual()` comprueba si dados dos números, estos son iguales o no.

2.4.3. Operadores Aritméticos

2.4.4. Operadores de desplazamiento

2.4.5. Acceso a Arrays

2.4.6. Expresiones condicionales

2.4.7. Expresiones condicionales

2.5. Ejemplo con constraints generadas automáticamente

Implementación

3.1. Explicación y ejemplo de la construcción de las reglas

Experimentos

- 4.1. Aplicación de la generación automática sobre juegos simples
- 4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints
- 4.3. Caso de uso, verificación y búsqueda de bugs